

Karmaşıklık Teorisi:

Dönemin bu noktasında algoritmik problemleri çözmek için “püf noktanızı” oluşturduk. Umarım bir problemle karşılaştığınızda, problemin nasıl çözüleceğine dair daha iyi bir fikriniz vardır. Hangi tasarım paradigması kullanılmalıdır (böl ve Yönet, DFS, açgözlü, Dönüştür-Yönet, dinamik programlama), ne tür veri yapıları alakalı olabilir (ağaçlar, yığınlar, graflar) ve hangi gösterimlerin en iyi olacağı (komşuluk listesi, komşuluk matrisleri)), algoritmanızın çalışma süresi nedir?

Probleminizi çözmek için kabul edilebilir derecede verimli bir algoritma keşfetmenize yardımcı olursa, tüm bunlar iyidir. Pratikte sıklıkla ortaya çıkan soru, kitaptaki her numarayı denemenizdir ve hiçbir şey işe yaramaz gibi görünüyor.

Algoritmanız, çözmek istediğiniz küçük problemleri ($n \leq 20$) makul şekilde verimli bir şekilde çözebilmesine rağmen gerçekten büyük uygulamalarda (örn. $N = 1000$ veya $n = 10.000$), algoritmanız asla sona ermez. Çalışma süresini analiz ettiğinizde, üstel zamanda yada daha kötü çalıştığını fark edersiniz!

$$n\sqrt{n}, 2^n, 2^{(2^n)}, n!$$

60'ların sonuna doğru, birçok kombinatoriyal problem için etkili çözümler bulmada büyük başarı elde edildi, ancak bilinen etkili bir algoritmik çözüm olmadığı görülen artan bir problem listesi de vardı.

İnsanlar, bu problemlerin çözümüne yol açacak bilinmeyen bir paradigmanın ya da belki de bu problemlerin doğal olarak çözülmesinin zor olduğunun ve üstel zamanda çalışan algoritmik çözümlerin bulunup bulunmadığının bir kanıtı olup olmadığını merak etmeye başladılar.

60'ların sonuna doğru dikkate değer bir keşif yapıldı. Bu zor problemlerin birçoğu, herhangi birini polinom zamanında çözebilirseniz, o zaman hepsini polinom zamanında çözebileceğiniz anlamında ilişkilidir. Bu keşif **NP tam**lık kavramını doğurdu ve muhtemelen bilgisayar bilimindeki en büyük açık problemleri oluşturdu: **P = NP** mi?

Bu alan yep yeni bir başlangıç. Çünkü odaklanma noktası değişiyor. Amaç artık bir problemin bir algoritma sunarak etkili bir şekilde çözülebileceğini kanıtlamak değildir. Bunun yerine, bir problemin etkili bir şekilde çözülemeyeceğini göstermeye çalışacağız. Soru, bunun nasıl yapılacağıdır?

Kuralları ortaya koymak: Verimli olarak çözülebilen problemler sınıfını verimsizce çözülebilen problemlerden ayırmanın bir yoluna ihtiyacımız var. Bunu **polinom zamanında** çözülebilecek problemleri göz önünde bulundurarak yapacağız.

Algoritmalar tasarlarken, çeşitli kavramlarla informal mümkün olmuştur. Akıllı bir programcının eksik detayları doldurabileceği gerçeğinden faydalandık. Bununla birlikte, **bir şeyin verimli bir şekilde yapılamayacağını kanıtlama işi**, çok daha dikkatli bir şekilde ele alınmalıdır, çünkü birisinin kuralları mantıksız bir şekilde değiştirmesine izin verecek ve gerçekte yokken etkili bir çözüme sahip olduğunu iddia edecek herhangi bir “boşluk” bırakmamak önemlidir.

En kötü durum karmaşıklığını kullanarak algoritmaların çalışma süresini n :giriş boyutu olmak üzere, n^n bir fonksiyonu olarak ifade ettik. Farklı büyüklükler için girdi boyutunu farklı tanımladık, ancak en alt düzey, girişi makul bir şekilde verimli kodlama kullanarak temsil etmek için gereken bit (veya bayt) sayısıdır. Oldukça verimli bir kodlama ile, aynı bilgiyi sağlamanın çok daha kısa bir yolu olmadığını varsayabiliriz.

Örneğin, sayıları ikili yerine, birli gösterimle ifade ederek, $(11111111)_1 = (100)_2 = 8$ sayılarına yazabilirsiniz, ancak bu kabul edilemez derecede verimsiz olacaktır. Grafları, tüm döngülerini listeleterek oldukça verimsiz bir şekilde tanımlayabilirsiniz, ancak bu da kabul edilemez. Sayıların ikili veya daha yüksek bazı olarak ifade edildiğini ve grafların bitişiklik matrisleri veya bitişiklik listeleri kullanılarak ifade edildiğini varsayacağız.

Aritmetik işlemlerin verimli bir şekilde gerçekleştirilebileceği açıktır. Ayrıca sayılar üzerindeki işlemlerin sabit zamanda yapılabilirliğini varsaydık. Şu andan itibaren daha dikkatli olmalıyız ve aritmetik işlemlerin en az saklanan sayılarda kesinlik kadar zaman gerektirdiğini varsaymalıyız.

Şimdiye kadar gördüğümüz tüm algoritmalar, en kötü çalışma sürelerinin yukarıda n , girdi boyutundaki bazı polinomlar tarafından sınırlandırılması özelliğine sahipti. Bir polinom zaman algoritması $O(n^k)$ zamanında çalışan, k 'nin n 'den bağımsız bir sabit olduğu herhangi bir algoritmadır. **Bir problemin, onu çözen bir polinom zaman algoritması varsa, polinom zamanında çözülebileceği söylenir.**

Polinomlara benzemeyen $O(n \log n)$ gibi bazı fonksiyonlar, $O(n^2)$ ile sınırlıdır. Polinomlara benzeyen bazı fonksiyonlar polinomsal değildir. Örneğin, n büyüklüğünde bir graf ve k tamsayısının girdiği ve $O(n^k)$ zamanında çalışan bir algoritmanız olduğunu varsayalım. Bu bir polinom mu? Hayır, çünkü **k probleme bir girdi olduğundan**, kullanıcının $k = n$ 'yi seçmesine izin verilir, bu da çalışma süresinin n 'de bir polinom olmayan $O(n^n)$ olacağını ima eder. Önemli olan n 'den bağımsız bir sabit olması gerektir.

Elbette tüm polinom zaman algoritmalarının “verimli” olduğunu söylemek yanlıştır. Çalışma süresi $O(n^{1000})$ olan bir algoritma kesinlikle oldukça verimsizdir. Bununla birlikte, bir algoritma polinom zamanından (örneğin 2^n) daha kötü çalışırsa, n 'nin çok küçük değerleri dışında kesinlikle verimli değildir.

Karar problemleri:

Tartıştığımız problemlerin birçoğu bir formun optimizasyonunu içerir: en kısa yolu bulun, minimum maliyetli kapsama ağacı bulun, minimum ağırlık üçgenlemesini bulun. Daha teknik nedenlerden ötürü, tartışacağımız NP-tam problemlerin çoğu karar problemi olarak ifade edilecektir.

Çıktısı basit bir "evet" veya "hayır" ise bir probleme karar problemi denir (veya bunu Doğru / Yanlış, 0/1, kabul / reddet olarak düşünebilirsiniz).

Karar problemleri:

Birçok optimizasyon problemini Karar problemleri açısından ifade edeceğiz. Örneğin, minimum kapsama ağaç karar problemi şunlar olabilir: Ağırlıklı bir G grafi ve bir k tamsayısı verildiğinde, G 'nin ağırlığı en fazla k olan bir kapsama ağacı var mı?

Bu, problemin daha az ilginç bir formülasyonu gibi görünebilir. Burada minimum kapsama ağacının ağırlığını istenmiyor ve bu ağırlığa ulaşan kapsama ağacının kenarlarını bile istemez. Burada yapılması istenen, bazı problemlerin **etkili bir şekilde çözülemeyeceğini göstermek** olacaktır. Basit karar probleminin etkili bir şekilde çözülemediğini gösterirsek, daha genel optimizasyon problemi de kesinlikle verimli bir şekilde çözülemez.

Dil Tanıma Problemleri:

Bir karar probleminin bir dil tanıma problemi olarak da düşünülebileceğini gözlemleyin. L dilini tanımlayabiliriz.

$$L = \{(G, k) \mid G \text{ has a MST of weight at most } k\}.$$

Bu küme çiftlerden oluşur, ilk eleman bir graftır. Bir katar olarak kodlanmış komşuluk matrisi) ve bunu izleyen ikili sayı olarak kodlanmış bir k tamsayıdır. İlk başta bir grafi dize olarak ifade etmek tuhaf görünebilir, ancak açıkçası bir bilgisayarda temsil edilen her şey bir şekilde bir bit dizisine bölünür.

Bir giriş katarı (G, k) sunulduğunda, Eğer $(G, k) \in L$, G 'nin en fazla k ağırlıklı bir kapsama ağacı olduğunu belirten EVET, aksi takdirde “HAYIR” yanıtı verecektir. İlk durumda algoritmanın girişi “kabul ettiğini”, aksi takdirde girişi “reddettiğini” söylüyoruz.

Herhangi bir dil göz önüne alındığında, belirli bir katarın dilde olup olmadığını belirlemenin ne kadar zor olduğu sorusunu sorabiliriz. Örneğin, MST dili L söz konusu olduğunda, üyeliği polinom zamanında kolayca belirleyebiliriz. Grafi dahili olarak saklarız, Kruskal'ın algoritmasını çalıştırırız ve nihai optimal ağırlığın en fazla k olup olmadığını görürüz. Geçerli ise **kabul** ediyoruz, aksi takdirde **reddediyoruz**.

Tanım: P'yi, üyeliğin polinom zamanında test edilebileceği tüm diller kümesi olarak tanımlayın. (Sezgisel olarak, bu, polinom zamanında çözülebilecek tüm karar problemlerine karşılık gelir.)

Dillerin dize kümeleri olduğunu ve P'nin bir dil kümesi olduğunu unutmayın. P, dilde tanınan üyeliğin hesaplanmasının ne kadar zor olduğu ile tanımlanır.

Üyeliği belirlemenin ne kadar zor olduğu ile tanımlanan dil grubuna karmaşıklık sınıfı denir.

Minimum kapsama ağaçları polinom zamanda hesaplayabildiğimiz için $L \in P$

$$M = \{(G, k) \mid G \text{ has a simple path of length at least } k\}$$

Bir grafi G ve tamsayı k verildiğinde, M dilinde olup olmadığını nasıl “tanıyabilirsiniz”? En az k uzunluğundan birini bulana kadar grafta basit yollar aramayı deneyebilirsiniz. Birini bulursanız kabul edebilirsiniz ve sonlandırabilirsiniz. Ancak, değilse, arama yapmak için çok zaman harcayabilirsiniz (özellikle k , $n - 1$ gibi büyükse, ve böyle bir yol yoksa).

Öyleyse $M \in P$ mi? Kimse cevabı bilmiyor. Aslında, M'nin NP-tam olduğunu göstereceğiz.

Aşağıda, bir dizi sınıf tanıtacağız. “Dil” ve “karar problemleri” terimleri arasında ileri geri atlayacağız, ancak bizim amacımız için aynı şeyleri kastediyorlar. Tüm teknik tanımları vermeden önce, genel sınıfların sezgisel düzeyde neye benzediğini biraz söyleyelim.

P: Polinom zamanında çözülebilen tüm karar problemleri kümesidir. Genel olarak bu problemlerden **“kolay” veya “etkin bir şekilde çözülebilir”** olarak bahsedilir.

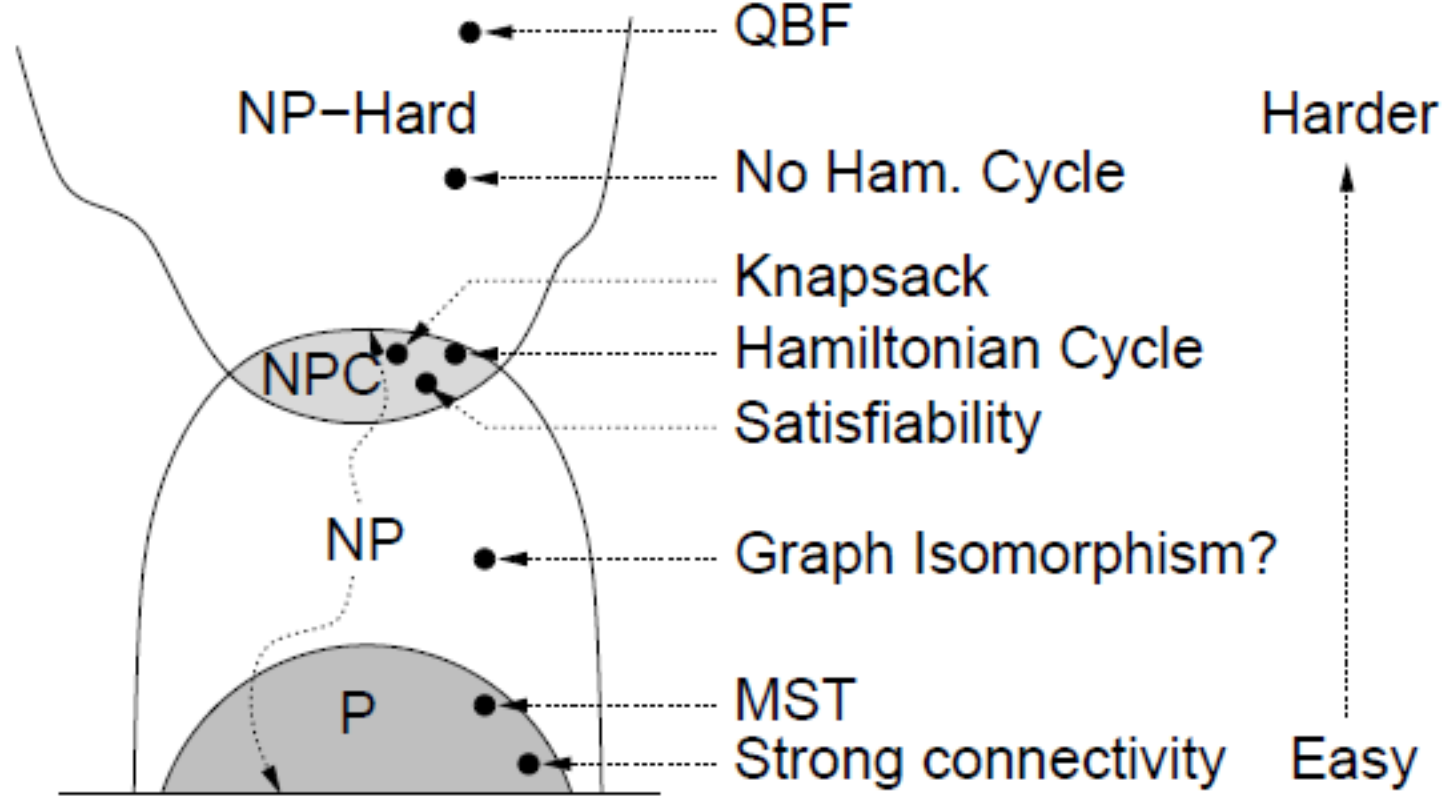
NP: Polinom zamanında **doğrulanabilen** tüm karar problemleri setidir. Bu sınıf, bir alt küme olarak P’yi içerir. Bu nedenle, bir dizi kolay problem içerir, ancak çözülmesinin çok zor olduğuna inanılan bir takım problemler de içerir. NP terimi “polinom değil” anlamına gelmez. Bu terim **“Non-Deterministic polinom zamanlı”** anlamına gelmektedir.

NP-zor: Problemin NP-hard olduğunu söylemek çözmenin zor olduğu anlamına gelmez. Daha ziyade, bu problemi polinom zamanında çözebilirsek, tüm NP problemlerini polinom zamanında çözebiliriz. Bir problemin NP zor olması için NP sınıfında olması gerekmediğini unutmayın. Tüm NP problemlerinin polinom zamanında çözülemeyeceğine inanıldığından, polinom zamanında NP-zor bir problemin çözülemeyeceğine inanılmaktadır.

NP-complete: Eğer bir problem NP ise **VE** NP-zor ise bu problem **NP-tam** sınıfındadır. Yani, **$NPC = NP \cap NP\text{-zor}$** .

Aşağıdaki şekilde P, NP, NP-zor ve NP-tam (NPC) setlerinin görünebileceği bir yol gösterilmektedir. Belki de diyoruz çünkü bu karmaşıklık sınıflarının hepsinin farklı olup olmadığını veya hepsinin polinom zamanında çözülebilir olup olmadığını bilmiyoruz. Şekilde tartışmayacağımız bazı problemler var. Birincisi, iki grafiğin köşelerinin yeniden adlandırılmasına kadar aynı olup olmadığını soran graf izomorfizmi'dir. Bu problemin NP'de olduğu bilinmektedir, ancak P'de olduğu bilinmemektedir.

Graf teorisine (graph theory) göre iki şekil birbirinden farklı çizilmiş ancak işlev ve değer olarak aynı olabilir. Bu durumda iki graf için izomorfik denilebilir.



Diğeri, Nicelenmiş Boole Formüllerini temsil eden QBF'dir. Bu problemde, nicelik belirteçli (9 ve 8) bir boole formülü verilir ve formülün doğru veya yanlış olup olmadığını bilmek istersiniz.

Polynomial Time Verification and Certificates:

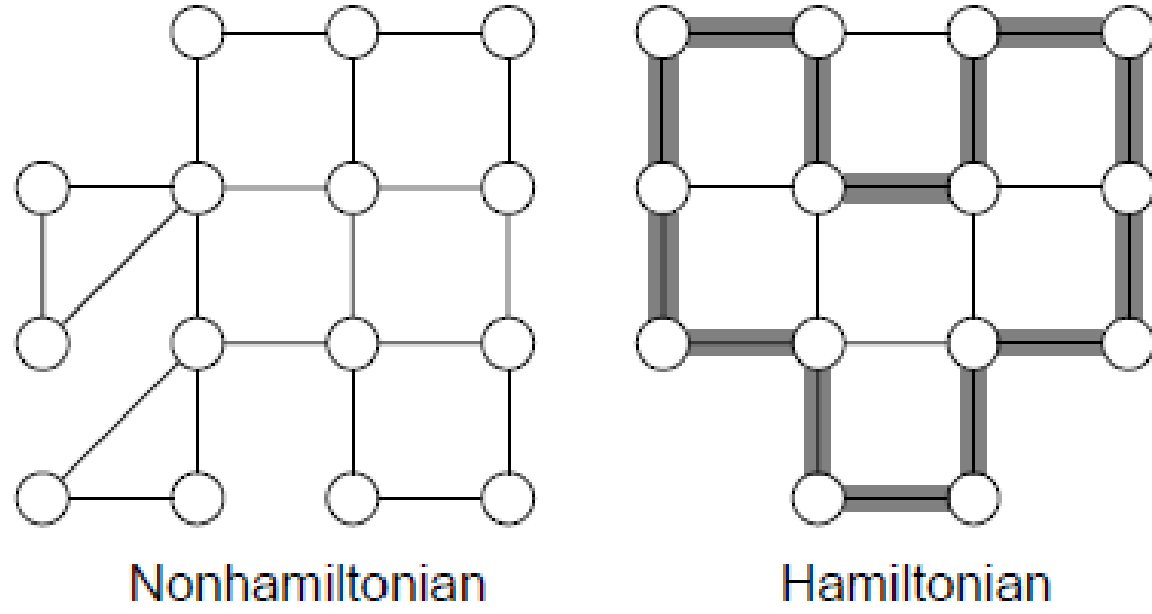
NP-complete problemlerinin sınıfı hakkında konuşmadan önce, bir doğrulama algoritması kavramını tanıtmak önemlidir. Çözülmesi çok zor olabilecek birçok dil tanıma problemi vardır, ancak bir katarın bu dilde olup olmadığını doğrulamanın kolay olduğunu söyleyebiliriz.

Hamilton döngüsü problemi olarak adlandırılan aşağıdaki problemi düşünün. Yönlendirilmemiş bir G grafiği verildiğinde, **G her köşeyi tam olarak bir kez ziyaret eden bir döngüye sahiptir.** (Yönlendirilmiş graflarda benzer bir problem var ve ayrıca tüm köşeleri ziyaret eden bir yol olup olmadığını soran bir sürüm var.) Bu problemi bir dil tanıma problemi olarak tanımlayabiliriz.

$$HC = \{ (G) \mid G \text{ has a Hamiltonian cycle} \}$$

burada (G) , bir G grafinin bir katar olarak kodlanmasını belirtir.

Hamilton döngüsü problemi çok daha zor görünüyor ve bu problem için bilinen bir polinom zaman algoritması yok. Örneğin, aşağıdaki şekil, biri Hamiltonyan diğeri de olmayan iki grafiği göstermektedir.



Bununla birlikte, bir grafin Hamilton döngüsüne sahip olduğunu varsayalım. O zaman birisinin bizi bu konuda ikna etmesi çok kolay bir mesele olur. $\langle v_3, v_7, v_1, \dots, v_{13} \rangle$

Daha sonra grafi inceleyebilir ve bunun gerçekten yasal bir döngü olduğunu ve grafiğin tüm köşelerini tam olarak bir kez ziyaret ettiğini kontrol edebiliriz.

Bu nedenle, Hamiltonian döngü problemini çözmenin etkili bir yolunu bilmememize rağmen, belirli bir grafın HC'de olduğunu doğrulamanın çok etkili bir yolu vardır. Verilen döngüye doğrulama denir. Bu, belirli bir katarın bir dilde olduğunu doğrulamamıza izin veren bir bilgi parçasıdır.

Daha formal olarak, bir L dili ve $x \in L$ verildiğinde, bir doğrulama algoritması, verilen x ve doğrulama olarak adlandırılan y dizesi olmak üzere, ve bu doğrulamayı yardım olarak kullanarak x 'in L dilinde olduğunu doğrulayabilir. x , L dilinde değilse doğrulanacak hiçbir şey yoktur. Tüm dillerin doğrulanması kolay özelliklere sahip olmadığını unutmayın. Örneğin, aşağıdaki dilleri göz önünde bulundurun:

$$\begin{aligned} UHC &= \{(G) \mid G \text{ has a unique Hamiltonian cycle}\} \\ \overline{HC} &= \{(G) \mid G \text{ has no Hamiltonian cycle}\}. \end{aligned}$$

Bir G grafının UHC dilinde olduğunu varsayalım. Birisi bize G 'nin gerçekten dilde olduğunu doğrulamamıza izin verecek hangi bilgileri verir? Bize benzersiz Hamilton döngüsünün bir örneğini verebilirler ve bunun Hamilton dönemi döngüsü olduğunu doğrulayabiliriz, ancak bizim **bunun tek olduğuna ikna olmamız** için ne tür bir doğrulama verebilirler?

Hamiltonian DEĞİL başka bir döngü verebilirler, ancak bu, Hamiltonian'ın bir yerinde başka bir döngü olmadığı anlamına gelmez. N uzunluğundaki diğer tüm döngüleri listelemeye çalışabilirler, ancak bu hiç verimli olmayacaktır, çünkü olası $n!$ kadar döngü var. Bu nedenle, birinin bize belirli bir grafın dilde olduğuna dair kendimizi tam olarak ikna edecek bazı bilgiler verebileceğini hayal etmek zor.

NP:

Tanım: NP'yi, bir polinomsal zaman algoritması ile **doğrulanabilecek** tüm diller kümesi olarak tanımlayalım.

Bu küme neden “VP” yerine “NP” olarak adlandırılıyor? Orijinal NP terimi “belirsiz olmayan polinom zamanı” anlamına geliyordu. Bu, tahmin yapabilen belirsiz olmayan bir bilgisayarda çalışan bir programa atıfta bulunur. Temel olarak, böyle bir bilgisayar doğrulamanın değerini belirsiz bir şekilde tahmin edebilir ve daha sonra dizinin polinom zamanında dilde olduğunu doğrulayabilir.

Belirsizlik konusu Biçimsel Diller ve Soyut Makineler dersinde işlenmişti.

P gibi NP de bazı karmaşıklık ölçütlerine (doğrulama karmaşıklığı) dayanan bir dildir. $P \subseteq NP$ 'yi gözlemleyin. Başka bir deyişle, bir problemi polinom zamanında çözebilirsek, polinom zamanında üyeliği kesinlikle doğrulayabiliriz. (Daha formal olarak, problemi çözmek için bir doğrulama görmemiz bile gerekmez, yine de polinom zamanda çözebiliriz).

Ancak $P = NP$ olup olmadığı bilinmemektedir. Bunun böyle olması gerektiğini düşünmek mantıksız görünüyor. Başka bir deyişle, **doğru bir çözümünüz olduğunu doğrulayabilmeniz, gerçek çözümü bulmanıza yardımcı olmaz**. Çoğu uzman $P \neq NP$ olduğuna inanıyor, ancak kimsenin buna ilişkin bir kanıtı yok.

NP-Completeness: Reductions

Karar Problemleri: Cevabının **evet** ya da **hayır** olduğu problemleri idi. NP-tam problemleri karar problemleri olarak ifade edilir ve bu nedenle girdinin bir katar olarak kodlandığı varsayılarak dil tanıma problemleri olarak düşünülebilir. Girdileri katar olarak kodlarız. Örneğin:

$$\begin{aligned}\text{HC} &= \{G \mid G \text{ has a Hamiltonian cycle}\} \\ \text{MST} &= \{(G, x) \mid G \text{ has a MST of cost at most } x\}.\end{aligned}$$

P: Polinom zamanında çözülebilen tüm karar problemlerinin sınıfıdır, bazı sabit k için $O(n^k)$. Örneğin $MST \in P$, ancak HC 'nin P 'de olduğu bilinmemektedir (ve şüphelenilmediğinden).

Doğrulama: Polinom zamanında bir katarın belirli bir dilde olduğunu doğrulamamıza izin veren bir kanıt parçasıdır. Örneğin, dilin Hamilton graflarının kümesi olduğunu varsayalım. Birisini bir grafiğin bu dilde olduğuna ikna etmek için, döngü boyunca bir dizi köşe içeren doğrulamayı sağlayabiliriz. Bunun G 'de geçerli bir döngü olduğunu belirlemek için bitişiklik matrisine erişmek kolaydır. Bu nedenle $HC \in NP$.

NP: Polinom zamanında doğrulanabilen tüm dillerin sınıfı olarak tanımlanır. P 'deki tüm diller polinom zamanında çözülebildiğinden, kesinlikle polinom zamanında doğrulanabileceğinden, $P \subseteq NP$ kabul edilmektedir. Bununla birlikte, NP , HC gibi çözülmesi gereken oldukça zor problemlere sahip gibi görünmektedir.

İndirgemeler (Reductions): NP -tam problemlerinin sınıfı, kimsenin nasıl verimli bir şekilde çözüleceğini bilmediği bir dizi karar probleminden (diller) (NP sınıfının bir alt kümesi) oluşur, ancak tek bir NP - için bile polinomsal zaman çözümü olan tam bir problem varsa, NP 'deki her problem polinom zamanında çözülebilir. Bunu kurmak için azaltma (Reduction) kavramını tanıtmamız gerekiyor.

İndirgemeleri tartışmadan önce, aşağıdaki soruyu ele alalım. Diyelim ki **H** ve **U** olmak üzere iki problem var. **H**'nin zor olduğunu biliyoruz (veya en azından buna inanıyoruz), yani polinom zamanında çözülemez. Öte yandan, **U**'nun karmaşıklığı bilinmemektedir, ancak bunun da zor olduğundan şüpheleniyoruz. **U**'nun polinom zamanında çözülemeyeceğini kanıtlamak istiyoruz. Bunu nasıl yaparız? Bunu göstermek istiyoruz

$$(H \notin P) \Rightarrow (U \notin P).$$

Bunu yapmak için, tartışmalı olanı kanıtlayabiliriz,

$$(U \in P) \Rightarrow (H \in P)$$

Başka bir deyişle, **U**'nun polinom zamanında çözemediğini göstermek için, polinom zamanında **U**'yu çözen bir algoritma olduğunu varsayalım ve sonra **H**'nin polinom zamanında çözülebileceğini göstererek bir çelişki türeteceğiz.

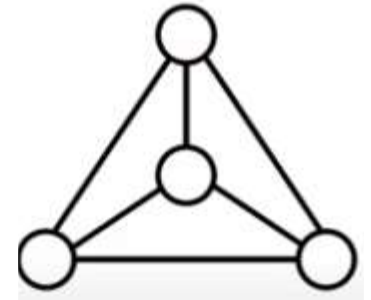
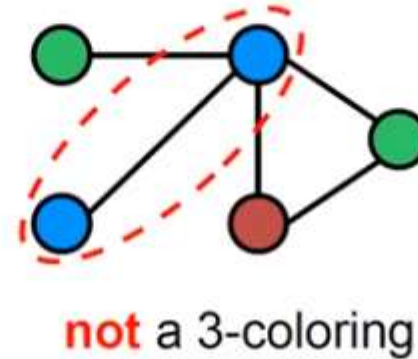
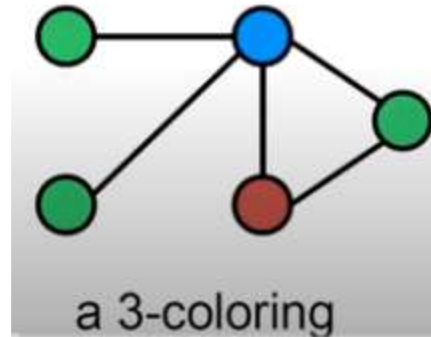
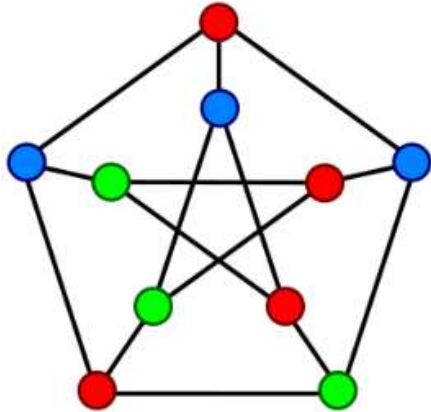
Bunu nasıl yapabiliriz? Varsayalım ki polinom zamanında **U** probleminin herhangi bir örneğini çözebilecek bir alt rutine sahibiz. O zaman tek yapmamız gereken bu alt rutini polinom zamanında **H** problemini çözmek için kullanabileceğimizi göstermektir. Bu nedenle, **H** problemini **U** problemine “**indirgedik**”. Burada, sözde alt programın gerçekten bir hayal olduğunu belirtmek önemlidir. **H**'nin polinom zamanında çözülemeyeceğini biliyoruz (veya kuvvetle inanıyoruz), bu nedenle esasen alt rutinin var olamayacağını kanıtlıyoruz, bu da **U**'nun polinom zamanında çözülemeyeceğini ima ediyor. (Bunu anladığınızdan emin olun, bu tüm **indirimler**in arkasındaki temeldir.)

3-Colorability and Clique Cover (3-Renklendirilebilirlik ve Klips Kapağı)

Bunu daha açık hale getirmek için bir örnek ele alalım. Aşağıdaki problemin NP-tam olduğu bilinmektedir ve bu nedenle problemin polinom zamanında çözölemeyeceğine inanılmaktadır.

Bir graf G verildiğinde, köşelerinin her biri 3 farklı "renkten" biriyle etiketlenebilir, böylece iki bitişik köşenin aynı etiketi olmaz.

Renklendirme, bölümün aynı kümesine iki nesnenin atanamayacağı bir kısıtlamanın olduğu çeşitli bölümlendirme problemlerinde ortaya çıkar. "Renklendirme" terimi, harita çizimindeki orijinal uygulamadan gelmektedir. Ortak bir sınırı paylaşan iki ülke farklı renklerle renklendirilmelidir. Düzlemsel grafların 4 renkle renklendirilebileceği iyi bilinir ve bunun için bir polinomsal zaman algoritması vardır. Ancak 3 rengin mümkün olup olmadığını belirlemek (düzlemsel graflar için bile) zor görünüyor ve bilinen bir polinomsal zaman algoritması yok. Aşağıdaki şekilde iki graf veriyoruz, biri 3 renklidir ve diğeri değildir.



Yönlendirilmemiş bir grafik G 'nin k -renklendirmesi, düğümlere renklerin atanmasıdır, böylece her bir düğümün tüm komşularından farklı bir renk atanır ve en fazla k renk kullanılır.

3Col problemi, polinom zamanında çözülemediğinden şüphelendiğimiz zor problem H 'nin rolünü oynayacaktır. Bilinmeyen problemimiz U için aşağıdaki problemi göz önünde bulundurun. Bir graf $G = (V; E)$ verildiğinde, $V' \subseteq V$ köşelerinin bir alt kümesinin, her $u, v \in V'$, $(u; v) \in E$ köşe çifti için bir klik oluşturduğunu söylüyoruz;. Yani V' ile indüklenen alt-graf tam bir graftir.

Bir probleme yönelik bir çözümün doğru olup olmadığını kontrol etmek kolaysa, problemi çözmek de kolay mıdır?

Bu, P'ye karşı NP probleminin özüdür. NP problemlerine tipik olarak Hamilton Yolu problemi gelmektedir: Ziyaret edilecek N şehir verildiğinde, bir şehri iki kez ziyaret etmeden bütün şehirleri nasıl ziyaret edebilirim? Bana bir çözüm vererseniz, doğru olup olmadığını kolayca kontrol edebilirim. Ama bu kadar kolay bir çözüm bulamıyorum.

NP-Completeness (NP-tam)

- Şu ana kadar herşey güzeldi!
 - Pek çok problem hızlı bir biçimde çözülebiliyor. (doğrusal zamana yakın veya en azından giriş boyutunun küçük polinom fonksiyonu olan bir zamanda)
- NP-completeness:Kötü haber
 - Birçok önemli sorunun hızlı bir şekilde çözülemeyeceğine dair kanıtlar.
- NP-complete problemler her yerde olabilir!

Dikkatli olmalıyız?

- Zor olduklarını bilmek, gereksiz çaba harcamamızı engeller...
 - **Sezgisel Yöntem:** yaygın vakaların makul bir kısmını çözmek için bir yöntem bulmak.
 - **Yaklaşık bir çözüm bulmak:** Doğruya yakın olduğunu kanıtlayabileceğiniz bir çözüm bulmak.
 - **Exponential Çözüm:** Problemi tam olarak çözmeniz ve daha iyi bir çözüm bulma konusunda endişelenmeyi bırakmanız gerekiyorsa.

Optimizasyon & Karar Problemleri

- **Karar problemleri**
 - Evet/Hayır
- **Optimizasyon problemleri**
 - En iyi değerli çözümün bulunması
- Optimizasyon problemleri, araştırılması daha kolay karar problemleri olarak kullanılabilir
 - *Örnek:* En kısa yol: G = ağırlıksız yönlü graf
 - U ve v arasında en az kenar kullanan bir yol bulun
 - U 'dan v 'ye en fazla k kenarından oluşan bir yol var mı?

Algoritmik vs Problem Karmaşıklığı

- Bir **hesaplamanın algoritmik karmaşıklığı**, hesaplamanın gerçekleştirilmesinin ne kadar zor olduğunun bir ölçüsüdür.
- **Hesaplama probleminin veya görevinin karmaşıklığı**, bu problemi çözmek veya o görevi yerine getirmek için en düşük karmaşıklıkta büyüme algoritmasına sahip algoritmanın karmaşıklığıdır. (*Ign time complexity*)
- **Hesapsal Karmaşıklık**: Problemlerin ne kadar zor olduklarına göre sınıflandırılması ile ilgilenir.

“P” Problemleri Sınıfı

- **P sınıfı**, polinomsal zamanda çözülebilen karar problemlerini içerir.
- Polinomsal-zaman algoritmaları
 - Worst-case running time: $O(n^k)$; k , sabit
- Polinomsal zaman örnekler:
 - $O(n^2)$, $O(n^3)$, $O(1)$, $O(n \lg n)$
- Non-polynomial zaman örnekler:
 - $O(2^n)$, $O(n^n)$, $O(n!)$

İzlenebilen/İzlenemez Problemler

- P sınıfındaki problemler, izlenebilen (**tractable**) Problemlerdir.
- P sınıfında olmayan **izlenemez (intractable)** yada **çözülemez (unsolvable)** problemlerdir.
 - Sadece küçük boyutta girişler için makul bir sürede çözülebilir. Yada hiçbir zaman çözülemezler.
- NP algoritmalar her zaman P'lerden kötü mü?
 - $n^{1,000,000}$:Teknik olarak «**tractable**», gerçekte imkansız
 - $n^{\log \log \log n}$:Teknik olarak «**intractable**», fakat kolay.

Çözülemez Problem

- Turing 1930'larda herhangi bir algoritma tarafından çözülemeyen problemlerin olduğunu keşfetti.
- En meşhurlarından biri: ***halting problem***
 - Rasgele bir algoritma ve girdisi göz önüne alındığında, bu algoritma sonunda duracak mı yoksa sonsuza kadar “sonsuz döngüde” devam edecek mi?”

İzlenemez Problemler Örnekler

Hamiltonian Paths

Optimization Problem: Given a graph, find a path that passes through every vertex exactly once

Decision Problem: Does a given graph have a Hamiltonian Path ?

Traveling Salesman

Optimization Problem: Find a minimum weight Hamiltonian Path

Decision Problem: Given a graph and an integer k , is there a Hamiltonian Path with a total weight at most k ?

İzlenemez Problemler

- Zorluk derecelerine göre çeşitli kategorilerde sınıflandırılabilir,
 - NP
 - NP-complete
 - NP-hard...

Nondeterministic and NP Algorithms

Nondeterministic algoritma = iki aşamalı yordam:

1) Nondeterministic (“tahmin”) aşaması:

aday çözüm olarak düşünülebilecek rastgele bir dize oluşturmak (“sertifika”)

2) Deterministic (“doğrulama”) aşaması:

sertifikayı ve örneği probleme götürür ve sertifika bir çözümü temsil ediyorsa EVET'i döndürür

NP algoritmalar (Nondeterministic polynomial)

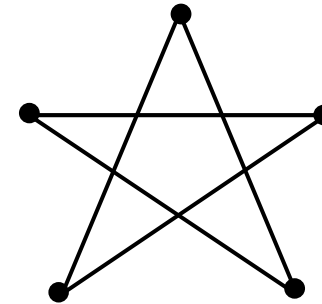
Doğrulama aşaması polinomsaldır.

“NP” Problemler sınıfı

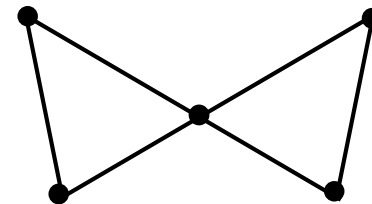
- **NP**:NP algoritmalarla çözülebilen problemleri içerir.
 - Polinomsal zamanda doğrulanabilir.
- Bize bir çözümün “sertifikası” verildiyse, sertifikanın zamanda girdi boyutuna göre polinom olarak doğru olduğunu doğrulayabiliriz.

Örnek: Hamiltonian Cycle

- **Giriş:** $G = (V, E)$ yönlü grafında V 'deki her bir köşeyi içeren basit bir döngüyü belirlemek.
 - Her bir köşe sadece bir defa ziyaret edilecek
- **Sertifika:**
 - Liste: $\langle v_1, v_2, v_3, \dots, v_{|V|} \rangle$



hamiltonian

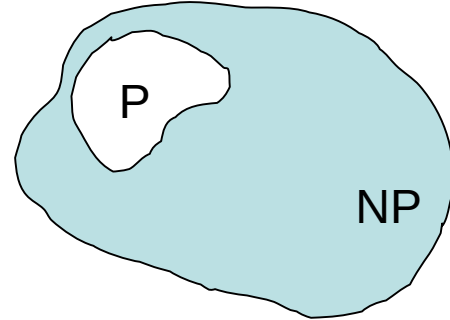


not
hamiltonian

P = NP?

- P'deki bir problem aynı zamanda

NP'de midir?: $P \subseteq NP$



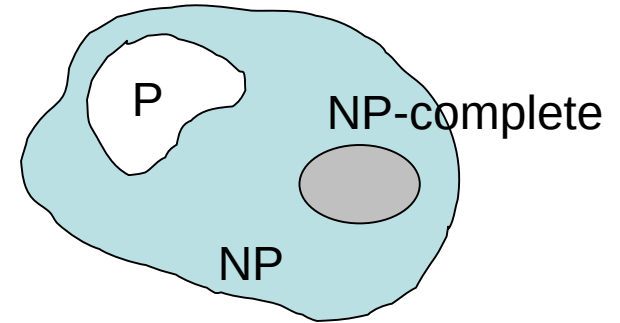
- Büyük soru: $NP \subseteq P$ or $P = NP$

– Bir çözümü kontrol etmek her zaman kolay ise, bir çözüm bulmak kolay mı olmalı?

- Çoğu bilgisayar bilimcisi bunun yanlış olduğuna inanıyor, ancak bir kanıtımız yok ...

NP-Completeness (informally)

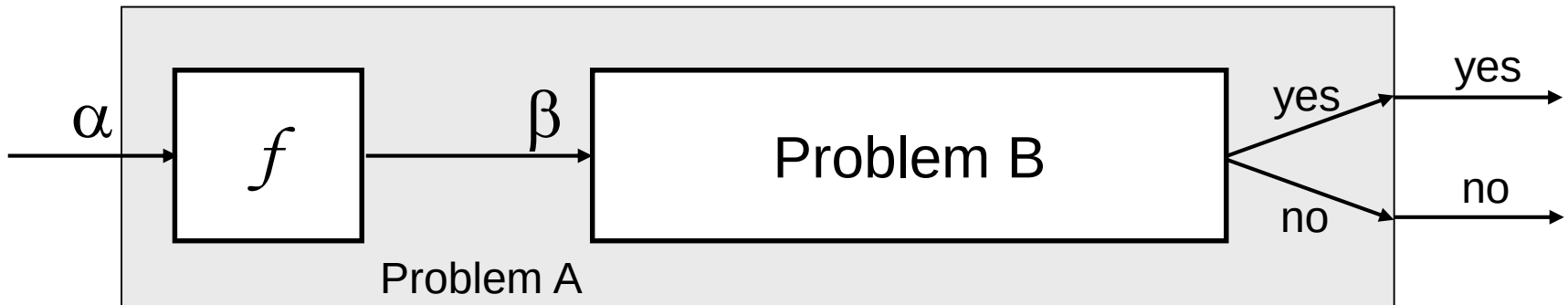
- **NP-complete** problemler, NP'deki en zor problemler olarak tanımlanır.



- Çoğu pratik sorun P veya NP-tam olarak ortaya çıkıyor.

Reductions (İndirgeme)

- İndirgeme, bir problemin diğerinden “daha kolay” olduğunu söylemenin bir yoludur.
- A probleminin B’den daha kolay olduğunu, (**“ $A \leq B$ ” biçiminde söyleriz**)
Eğer A’yı B’yi çözen bir algoritma ile çözebiliyorsak,
- Idea:** A’nın girişlerini B’nin girişlerine dönüştür



Polinomsal İndirgemeler

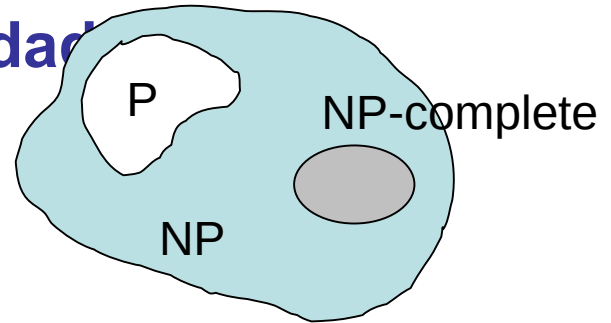
- Verilen A, B problemleri için, A polinomsal olarak B ($A \leq_p B$) indirgenebilir (**reducible**):
 1. Polinomsal zamanda A'nın girişlerini, B'nin girişlerine dönüştüren bir f fonksiyonu vardır.
 2. $A(i) = \text{YES} \Leftrightarrow B(f(i)) = \text{YES}$

NP-Completeness

- Bir B problemi **NP-complete sınıfındadır**

(1) $B \in \mathbf{NP}$

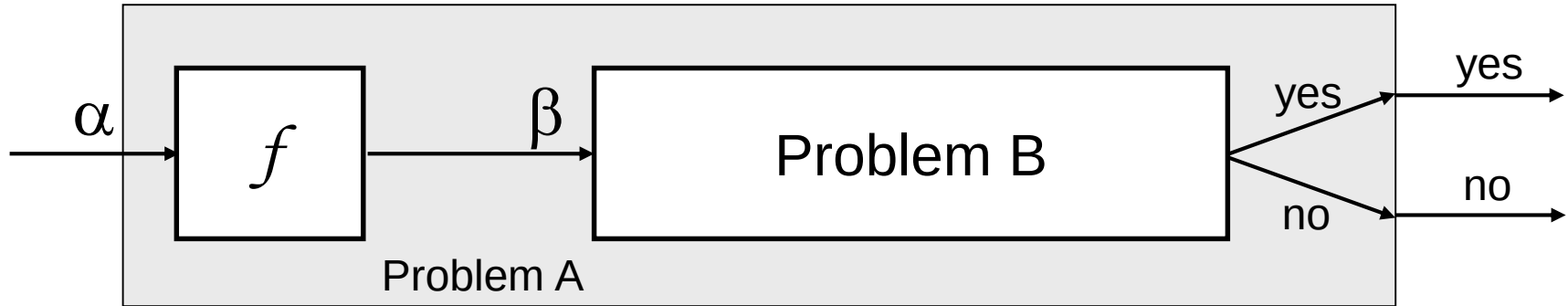
(2) Bütün $A \in \mathbf{NP}$ için $A \leq_p B$



Eğer B sadece özellik (2)'yi sağlıyorsa B, NP-hard sınıfındadır.

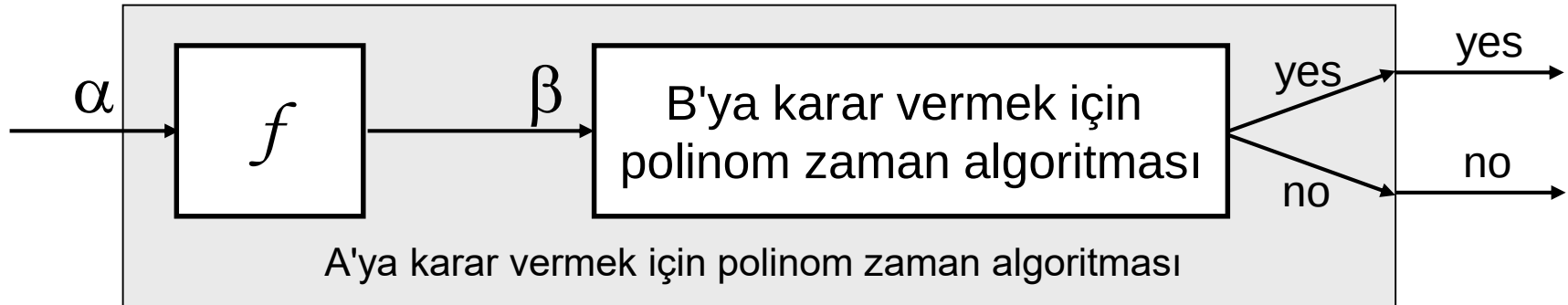
- NP zaman algoritması, **NP-Complete** problem için keşfedildi.
- Hiç kimse NP-Complete problemi için hiçbir polinom zaman algoritmasının mevcut olmadığını kanıtlamamıştır

Indirgeme (Reduction)



- If $A \leq_p B$ and $B \in P$, then $A \in P$
- if $A \leq_p B$ and $A \notin P$, then $B \notin P$

Polinom Zamanını Kanıtlamak

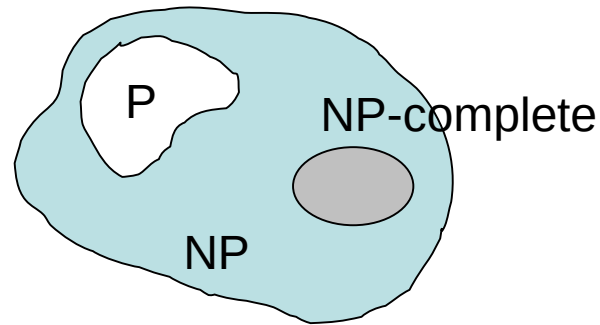


1. A'yı B'ye dönüştürmek için **polynomial time** reduction algoritması kullan
2. B için bilinen bir **polynomial time** algoritması kullan
3. B'nin cevabını A'nın cevabı olarak kullanın

Pratikte NP-Completeness'i ispatlamak

- B probleminin NP'de olduğunu ispatla:
 - Rasgele oluşturulan bir dize, bir çözümü temsil edip etmediğini belirlemek için polinom zamanında kontrol edilebilir.
- Bilinen bir NP-Complete probleminin polinom zamanında B'ye dönüştürülebildiğini gösterin
 - Tüm NP-Complete sorunlarının B'ye indirgenebilir olup olmadığını kontrol etmeye gerek yoktur

“Is $P = NP$?”



Theorem: Herhangi bir NP-Complete problem
polinomsal zamanda çözülebiliyorsa

$\Rightarrow$ then $P = NP$.

P & NP-Complete Problemler

- **En kısa basit yol**

- Verilen bir $G = (V, E)$ grafında kaynak düğümden diğer bütün düğümlere **en kısa** yolu bulmak
- Polnomsal çözüm: $O(VE)$

- **En uzun basit yol**

- Verilen bir $G = (V, E)$ grafında kaynak düğümden diğer bütün düğümlere **en kısa** yolu bulmak
- NP-complete

P & NP-Complete Problemler

- **Euler Halkası**

- $G = (V, E)$ bağlı, yönlü grafta her kenardan sadece bir defa geçmek üzere (bir düğüm çok defa ziyaret edilebilir), bir döngü yolu bulunur.
- Polinomsal çözüm $O(E)$

- **Hamiltonian halkası**

- $G = (V, E)$ bağlı ve yönlü grafta, G 'nin her düğümünü tam olarak bir defa ziyaret eden bir yolu bulur.
- NP-complete

Satisfiability Problem (SAT)

- **Satisfiability problem:** mantıksal bir Φ ifade verildiğinde, Φ 'nin T (TRUE) olarak değerlendirilmesine neden olan x_i değişkenlerine, bir değer ataması (F, T) nasıl olur?

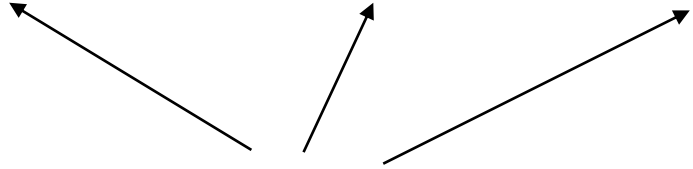
$$\Phi = x_1 \vee \neg x_2 \wedge x_3 \vee \neg x_4$$

- SAT, NP-complete olduğu gösterilen ilk problemidir.

CFN Satisfiability

- CFN, SAT'ın özel bir durumudur.
- Φ aşağıdaki durumlarda “Conjunctive Normal Form” (CNF)
 - “AND”
 - Her cümle değişkenlerin ve tümleyenlerinin yalnızca “OR” lanmasını içerir.

Örnek: $\Phi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2)$



clauses

3-CNF Satisfiability

CNF probleminin bir alt durumu:

– Üç cümle içerir

- *Örnek:*

$$\Phi = (x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_3 \vee x_2 \vee x_4) \wedge (\neg x_1 \vee \neg x_3 \vee \neg x_4)$$

- **3-CNF**, NP-Complete sınıfındadır.
- İlginçtir ki, **2-CNF**, **P** sınıfındadır.

Clique

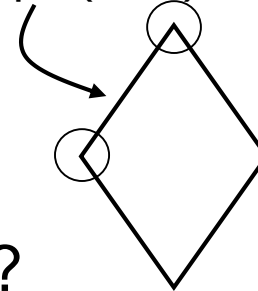
Clique Problem:

- $G = (V, E)$ yönsüz grafi
- **Clique:** Graftaki hepsi birbiriyle bağlı k düğüm içeren alt kümesi, (tam graf biçiminde)
- **Size of a clique:** içerdiği köşe sayısı

Optimizasyon problemi:

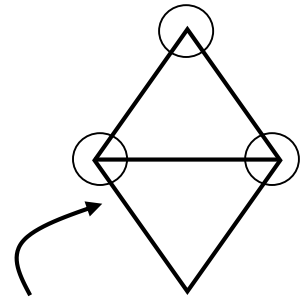
- Maksimum boyutta bir klik bulun

Clique($G, 2$) = YES
Clique($G, 3$) = NO



Karar problemi:

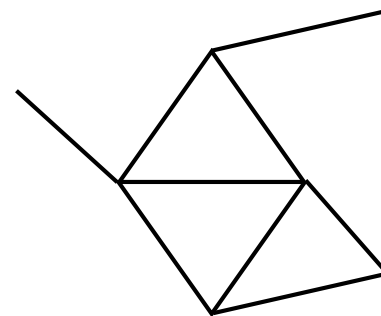
- G 'nin k boyutunda bir klik var mı?



Clique($G, 3$) = YES
Clique($G, 4$) = NO

Clique Doğrulayıcısı

- **Given:** $G = (V, E)$ yönsüz grafi
- **Problem:** G 'nin k boyutunda bir kliki var mı?
- **Certificate:**
 - Bir k düğümler kümesi
- **Verifier:**
 - Bu kümedeki tüm köşe çiftleri için E 'de bir kenar olduğunu doğrulayın.



3-CNF $\leq_p$ Clique

- **Idea:**

- G grafını G ancak G'nin k boyutunda bir klikine sahip olması durumunda doğrulayacak şekilde oluşturun

NP-İsimlendirme düzeni

- **NP-complete** – NP’de çözülmesi en zor problemler.
- **NP-hard** – En az NP kadar zordur. (NP’de olmak zorunda değil);
- **NP-easy** – en fazla NP kadar zor. (NP’de olmak zorunda değil);
- **NP-equivalent** –NP kadar zor, (NP’de olmak zorunda değil);

Examples NP-complete and NP-hard problems

Hamiltonian Paths

NP-complete

Optimization Problem: Given a graph, find a path that passes through every vertex exactly once

Decision Problem: Does a given graph have a Hamiltonian Path ?

Traveling Salesman

NP-hard

Optimization Problem: Find a minimum weight Hamiltonian Path

Decision Problem: Given a graph and an integer k , is there a Hamiltonian Path with a total weight at most k ?